

Question **1**

Correct

Mark 2.00 out of 2.00

Flag question

Mik a szimultán többszálú feldolgozást támogató processzorok jellemzői?

- IGAZ

✓

 Több regiszterkészlet van a processzorban
- HAMIS

✓

 Csak akkor vált szálát, ha az aktuális szál feldolgozása várakozni kényszerül
- IGAZ

✓

 Több utasításszámláló van a processzorban
- HAMIS

✓

 Egy adott órajelben csak egyetlen szál utasításait tudja végrehajtani

Question **2**

Partially correct

Mark 1.25 out of 2.00

Flag question

Mely állítások igazak a fizikai regiszterekre?

- HAMIS

✓

 A programozó ezeket, és nem a logikai/architekturális regisztereket használja
- IGAZ

✓

 A programozó elől el vannak takarva
- IGAZ

✓

 Több van belőle, mint a logikai/architekturális regiszterből
- IGAZ

✗

 Az utasításkészlet architektúra definiálja a darabszámukat és elnevezésüket

Question **3**

Partially correct

Mark 1.25 out of 2.00

Remove flag

Mik a virtuálisan indexelt, virtuális tag-eket használó cache tulajdonságai a fizikailag indexelt, fizikai tag-eket használó cache-hez képest?

- IGAZ

✓

 Kizárólag virtuális memóriát támogató processzorokban alkalmazható
- HAMIS

✗

 A címfordítás bizonyos esetekben elhagyható
- HAMIS

✓

 Rövidebbek a cache tag-ek
- HAMIS

✓

 Nagyobb méretű fizikai memória használatát támogatja

Question **4**

Correct

Mark 2.00 out of 2.00

Flag question

Hogyan hasznosítja a gshare prediktor a globális és lokális információkat az elágazásbecslés során?

- IGAZ

✓

 A globális előzmény regisztert és az utasításszámlálót XOR-olja
- HAMIS

✓

 Egy tisztán lokális és egy globális, korrelált becslő eredményét NXOR-olja
- HAMIS

✓

 A globális és a lokális becslő közül annak hisz, amelyik az utóbbi időben pontosabban becsül
- HAMIS

✓

 A gshare prediktor nem vegyít globális és lokális információkat

Question **5**

Incorrect

Mark 0.00 out of 2.00

Remove flag



Legfeljebb hány utasítás párhuzamos végrehajthatóságát tudja jelezni **a fordító** a processzornak?

	VLIW	EPIC	Szuperskalár
Annyit, ahány műveleti egysége van a processzornak	X ✓		
Amilyen mély a pipeline, annyit	X ✗ [blank]	X ✗ [blank]	
A fordító nem tudja jelezni, mely utasítások hajthatók végre párhuzamosan		X ✗ [blank]	X ✓

Question **6**

Correct

Mark 2.00 out of 2.00

Remove flag



A tanult 5 fázisú pipeline-ban mely fázisok **nem** végeznek tényleges munkát a Store és a Mul (szorzás) utasítás végrehajtásakor?

	Store	Mul
ID		
MEM		X ✓
WB	X ✓	
EX		

Question **7**

Incorrect

Mark 0.00 out of 2.00

Flag question

Mekkora a PHT mérete, ha az utasításszámláló utolsó 6 bitje jelöli ki a használt bejegyzést, és az ugrási hajlandóság nyomon követésére 4 állapotot különböztetünk meg?

A PHT mérete (bitben): ✗ [128]

Question **8**

Incorrect

Mark 0.00 out of 2.00

Flag question

Amdahl törvénye szerint mennyivel gyorsabban fut egy program egy 10 processzorból álló multiprocesszoros rendszerben, mint egy 1 processzoros rendszerben, ha a program harmada csak szekvenciálisan futtatható?

A gyorsulás mértéke: ✗ [2.5 or 2,5]

Question **9**

Complete

Mark 1.00 out of 2.00

Flag question

Mit jelent a pipeline átviteli sebessége?

Azt jelenti, hogy egyszerre, hány utasítás fejeződik be.

Comment:

Question 10

Correct

Mark 3.00 out of 3.00

Flag question

Legyen adott az alábbi utasítás sorozat:

```
i1: R1 ← R1 + R4
i2: R2 ← MEM[R0+4]
i3: R1 ← R1 + R2
i4: R3 ← R1 * R2
i5: MEM[R0+0] ← R3
```

Az utasítássorozatot egy 6 fokozatú utasítás pipeline-t használó processzor hajtja végre. A pipeline minden utasítás végrehajtását 5 részműveletre bontja: betölti (IF), dekódolja (ID), végrehajtja a vonatkozó aritmetikai műveletet (EX), majd a vonatkozó memóriaműveletet (MEM), végül a regisztertárolóba írja az eredményregiszter értékét (WB). Az IF, ID, EX és WB fázisok késleltetése 1 órajel, a MEM fázis késleltetése 2 órajel, de iterációs ideje 1 órajel (vagyis 2 pipeline fokozatként jelenik meg: ME0 és ME1). A címre, és store esetén az adatra az ME0 elején van szükség, load esetén pedig az ME1 végén jelenik meg az adat. Minden utasítás végrehajtása mind az 5 fázison átesik, függetlenül attól, hogy szüksége van-e rá. Minden forwarding út használata megengedett. Ha bármilyen egymásrahatás az utasítás megállítását igényli, az utasítás mindig a legutolsó olyan fázisban áll meg, ameddig egymásrahatás nélkül eljut.

Adja meg az utasítássorozat ütemezését (melyik utasítás mikor melyik fázisban van)! Ha szünetet kell beiktatni, jelezze, hogy mi az oka! Használja az alábbi jelöléseket:

- A*: a szünet oka adategymásrahatás
- F*: a szünet oka feldolgozási egymásrahatás
- P*: a szünet oka procedurális egymásrahatás

A megoldását írja az alábbi táblázatba! Minden egyes sor utolsó oszlopába írja be, ha az utasítás feldolgozása során forwarding-ra volt szükség, hogy melyik pipeline regiszterekből kell kiolvasni a kívánt értéket (ha több ilyen is van, vesszővel elválasztva, szóköz nélkül)! (3p)

	1.	2.	3.	4.	5.	6.	7.	8.	9.	10.	11.	12.	Forwarding
i1	IF	ID	EX	ME0	ME1	WB							
i2		IF	ID	EX	ME0	ME1	WB						
i3			IF	ID	A*	A*	EX	ME0	ME1	WB			ME1/WB
i4				IF	F*	F*	ID	EX	ME0	ME1	WB		EX/ME0

Question 11

Correct

Mark 1.00 out of 1.00

Remove flag

Rendezze át az utasítássorozatot úgy, hogy az a lehető leggyorsabban fusson le, gyorsabban, mint eredetileg! (1 pont)

Az átrendezett utasítássorozat: i2 ✓ - i1 ✓ - i3 ✓ - i4 ✓ - i5 ✓

Your answer is correct.

The correct answer is:

Rendezze át az utasítássorozatot úgy, hogy az a lehető leggyorsabban fusson le, gyorsabban, mint eredetileg! (1 pont)

Az átrendezett utasítássorozat: [i2] - [i1] - [i3] - [i4] - [i5]

Question 12

Partially correct

Mark 3.00 out of 4.00

Flag question

Egy processzorban az adat cache 4 utas asszociatív szervezésű, LRU menedzsel, 8 memóriablokkot képes tárolni.

A felhasználói program az alábbi memóriablokkokat hivatkozta (ebben a sorrendben):

- 7, 1, 5, 0, 12

(a) Adja meg a blokkokhoz tartozó cache index-eket, valamint a cache tag-eket! (4 pont)

	7	1	5	0	12
Index mező:	1 ✓	0 ✗ [1]	1 ✓	0 ✓	0 ✓
Cache tag:	3 ✓	0 ✓	2 ✓	0 ✓	6 ✓

Your answer is partially correct.

Legyen adott az alábbi utasítás sorozat:

```
i1: R1 ← MEM[R0+0]
i2: R2 ← MEM [R0+4]
i3: R2 ← R2 + R1
i4: R0 ← R0 + 8
```

Szüntesse meg a WAW és WAR adat-egymásrahatásokat regiszter átnevezés segítségével! A regiszter átnevezést végezze el minden utasításra szisztematikusan, ott is, ahol nem lenne rá szükség! A szükséges fizikai regisztereket jelölje U0, U1, U2, ... stb. Vegye figyelembe, hogy az alábbi regiszter leképző tábla a kezdeti állapotot is tartalmazza! Ha új fizikai regiszterre van szükség, válassza mindig a táblázatban szereplő legnagyobb után közvetlen következőt! Minden egyes programsor feldolgozása után frissítse a regiszter leképző táblát (csak a megváltozott bejegyzést kell beírni)! A mezőkbe sehova ne írjon szóköz karaktert! (3 pont)

Az utasítássorozat átnevezés után:

i1': U10 ✓ ← MEM[U9+0] ✓

i2': U11 ✓ ← MEM[U9+4] ✓

i3': U12 ✓ ← U11+U10 ✓

i4': U13 ✓ ← U9+8 ✓

Regiszter-leképző tábla:

	Kezdetben	i1	i2	i3	i4
R0:	U9				U13 ✓
R1:	U5	U10 ✓			
R2:	U8		U11 ✓	U12 ✓	
R3:	U1				

Ha van egy ideális out-of-order processzorunk, melyben a pipeline szélessége végtelen, és minden utasítást egyetlen órajel alatt feldolgoz, mennyi ideig tart a fenti utasítássorozat végrehajtása

Regiszterátnevezés nélkül: 2 ✓ órajel

Regiszterátnevezéssel: 2 ✓ órajel

(A processzor minden egyes órajelben minden olyan utasítást végre tud hajtani egyszerre, mely a függőségek figyelembe vételével lehetséges)

Your answer is correct.